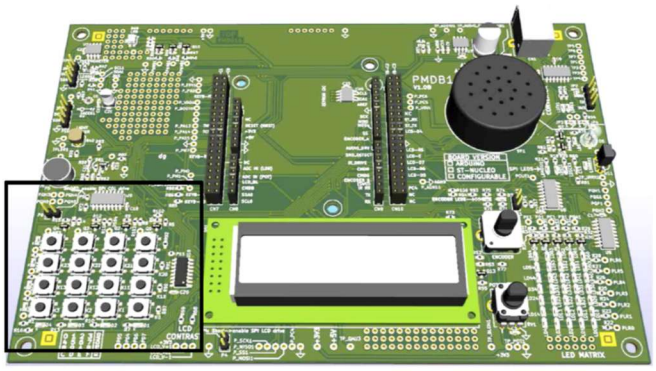
Lab10 - Keyboard

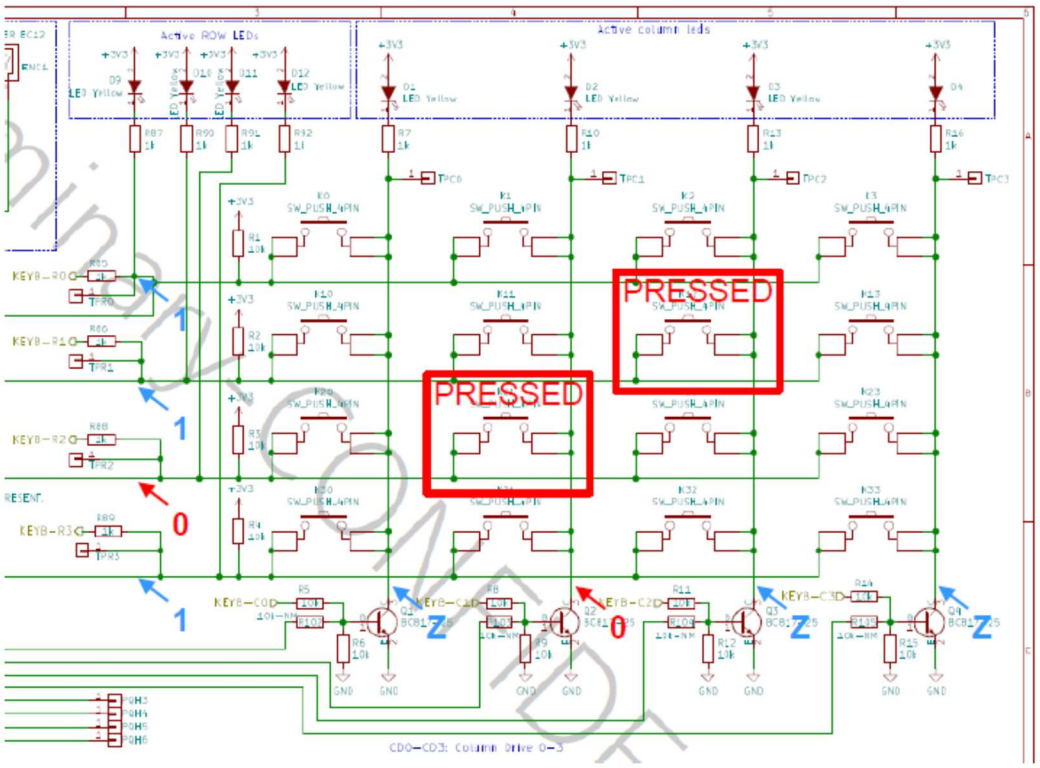
L'hardware: come è fatta la tastiera 4×4

La tastiera è una matrice **4×4 (16 tasti)** collegata secondo lo schema classico "keypad scan":



KEYBOARD

- **Righe (Row Readout)**: PC2, PC3, PC12, PC13 → configurate come **GPIO_Input**. Sulla scheda sono già presenti pull-up su queste linee, quindi **a riposo leggono sempre '1'**.
- **Colonne (Column Driver)**: PC8, PC9, PC10, PC11 → configurate come **GPIO_Output**. Ogni colonna non pilota direttamente il pin ma la base di un transistor **open collector**: quando il GPIO è alto il transistor è spento e la colonna resta "flottante" (alta impedenza, indicata con **Z** nello schema); quando il GPIO è basso il transistor si accende e tira la colonna a massa (GND).



Il principio di lettura (scanning)

Non puoi leggere tutti e 16 i tasti in un colpo solo con 8 pin: la tecnica è lo **scan a colonne**, una alla volta:

1. Abiliti **una sola colonna per volta** (GPIO di quella colonna a livello basso → transistor ON → colonna a massa); tutte le altre colonne restano disabilitate (GPIO alto → transistor OFF → colonna flottante/Z, non influenzano la lettura).
2. Leggi le **4 righe**:
   - riga a '1' (alta, per via del pull-up) → il tasto su quella riga/colonna **non è premuto** (nessun percorso verso massa);
   - riga a '0' → il tasto è **premuto** (il tasto ha chiuso il contatto verso la colonna che hai tirato a massa).
3. Passi alla colonna successiva e ripeti. Scansionando ciclicamente le 4 colonne copri tutti e 16 i tasti.
4. **Debounce**: un contatto meccanico rimbalza per qualche ms quando viene premuto/rilasciato. Prima di considerare valido un tasto, bisogna aspettare che resti premuto per una durata minima.

L'obiettivo del progetto è fare lo scan **periodicamente usando un timer a interrupt** invece di un while con HAL_Delay bloccante: ogni tot millisecondi scatta l'interrupt, cambi colonna e leggi le righe, mentre il while(1) nel main resta libero di fare altro (in questo caso: controllare se un tasto è stato tenuto premuto abbastanza a lungo e mandarlo via UART).

Stato per ciascuno dei 16 tasti, con una macchina a stati a 3 valori per rispettare esattamente la consegna ("non ripetere l'invio finché il tasto resta premuto, rimanda il codice se viene rilasciato e ripremuto"):

```
typedef enum { KEY_IDLE, KEY_DEBOUNCE, KEY_SENT } KeyState;

volatile KeyState key_state[16] = {KEY_IDLE};
volatile uint32_t press_start[16] = {0};
volatile uint8_t  current_col = 0;

#define DEBOUNCE_MS 30

GPIO_TypeDef* const COL_PORT = GPIOC;
const uint16_t COL_PIN[4] = {GPIO_PIN_8, GPIO_PIN_9, GPIO_PIN_10, GPIO_PIN_11};
GPIO_TypeDef* const ROW_PORT = GPIOC;
const uint16_t ROW_PIN[4] = {GPIO_PIN_2, GPIO_PIN_3, GPIO_PIN_12, GPIO_PIN_13};

const char keymap[4][4] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
```

Callback del timer (chiamata ogni pochi ms): scansiona la colonna corrente, aggiorna lo stato dei 4 tasti su quella colonna, poi passa alla colonna successiva.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2)
  {
    // 1. leggi le righe con la colonna CORRENTE già abilitata
```

```
    //   (l'ho abilitata all'interrupt precedente: il tempo di
    //   assestamento elettrico è di microsecondi, trascurabile
    //   rispetto al periodo del timer)
    for (int r = 0; r < 4; r++)
    {
      uint8_t idx = current_col * 4 + r;
      GPIO_PinState row = HAL_GPIO_ReadPin(ROW_PORT, ROW_PIN[r]);

      if (row == GPIO_PIN_RESET) // riga bassa -> tasto premuto
      {
        if (key_state[idx] == KEY_IDLE)
        {
          key_state[idx] = KEY_DEBOUNCE;
          press_start[idx] = HAL_GetTick();
        }
        // se già KEY_DEBOUNCE o KEY_SENT non tocco nulla qui:
        // la validazione del debounce la fa il main loop
      }
      else // riga alta -> tasto rilasciato
      {
        key_state[idx] = KEY_IDLE; // pronto per una nuova pressione
      }
    }

    // 2. disabilita la colonna corrente, abilita la prossima
    HAL_GPIO_WritePin(COL_PORT, COL_PIN[current_col], GPIO_PIN_SET); // OFF (Z)
    current_col = (current_col + 1) % 4;
    HAL_GPIO_WritePin(COL_PORT, COL_PIN[current_col], GPIO_PIN_RESET); // ON (GND)
  }
}
```

Nel while(1) del main: controlli il debounce e mandi il carattere via UART solo al momento del passaggio da KEY_DEBOUNCE a validato.

```
while (1)
{
  for (int i = 0; i < 16; i++)
  {
    if (key_state[i] == KEY_DEBOUNCE &&
      (HAL_GetTick() - press_start[i]) >= DEBOUNCE_MS)
    {
      char buf[8];
      int len = sprintf(buf, "%c\r\n", keymap[i / 4][i % 4]);
      HAL_UART_Transmit(&huart2, (uint8_t*)buf, len, HAL_MAX_DELAY);
      key_state[i] = KEY_SENT; // non reinviare finché non torna KEY_IDLE
    }
  }
}
```

Perché questo schema rispetta esattamente le richieste della traccia: KEY_SENT blocca i re-invii mentre il tasto resta premuto; solo quando la riga torna alta la callback riporta lo stato a KEY_IDLE, e alla pressione successiva si riparte da KEY_DEBOUNCE → nuovo invio. È lo stesso principio del debounce "adesso − ultimo

evento" che avevi già usato per l'EXTI del pulsante: qui uso HAL_GetTick() - press_start[i], mai l'ordine inverso (occhio a questo, con uint32_t un ordine sbagliato dà underflow e il debounce sparisce silenziosamente).

Configurazione in CubeMX

1. Parti dal progetto base come per gli altri lab.
2. Pin **PC8, PC9, PC10, PC11** → GPIO_Output (colonne). Output level iniziale: alto (colonne disabilitate a riposo), tranne la prima che abiliterai a mano prima di partire con il timer.
3. Pin **PC2, PC3, PC12, PC13** → GPIO_Input (righe). Pull-up già presente sulla scheda, ma non fa danno abilitare anche il pull-up interno (GPIO Mode: Input, Pull-up).
4. Configura un timer (es. TIM2) per generare interrupt periodico ogni pochi ms:
    - **Parameter Settings**: calcola Prescaler/Period per la frequenza di scan voluta.
    - **NVIC Settings**: spunta "TIM2 global interrupt".
5. USART già configurata per il lab (come nei precedenti) per l'invio bloccante HAL_UART_Transmit. Se invece vuoi usare _DMA, ricordati che ti serve **anche** la casella "USARTx global interrupt" in NVIC oltre al DMA request — altrimenti la trasmissione funziona una volta sola e poi resta bloccata su BUSY, esattamente come ti era successo nel Lab04/Lab06.
6. Genera il codice, poi nel main() prima di avviare il timer abilita la prima colonna:
7. HAL_GPIO_WritePin(GPIOC, GPIO_PIN_8, GPIO_PIN_RESET); // colonna 0 attivaHAL_TIM_Base_Start_IT(&htim2);

Debug consigliato

Come suggerisce la traccia stessa: prova a rallentare artificialmente lo scan (periodo del timer di qualche centinaio di ms invece di pochi ms) mentre debuggi, così puoi seguire a occhio/con breakpoint quale colonna è attiva e verificare riga per riga che la lettura corrisponda al tasto che stai fisicamente premendo, prima di velocizzare lo scan al valore definitivo.

Se vuoi, posso trasformare questa spiegazione in un documento salvato nel progetto così la ritrovi assieme agli altri materiali del corso.

Lab10

Lab10: scansiona ogni colonna e leggi la keyboard usando una timer interrupt

File .ioc

Pin

Riga	Pin	Configurazione
0	PC2	GPIO_Input
1	PC3	GPIO_Input
2	PC12	GPIO_Input
3	PC13	GPIO_Input

Colonna	Pin	Configurazione
0	PC8	GPIO_Output
1	PC9	GPIO_Output
2	PC10	GPIO_Output
3	PC11	GPIO_Output

Timer

- Timer 2 (4ms):
    - Clock Source: Internal Clock
    - TIM2 global interrupt: Enabled

- PSC=99
- ARR=3359

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(3359+1)(99+1)} = 250\ Hz$$

$$T_{Timer} = \frac{(ARR+1)(PSC+1)}{f_{Clock}} = \frac{(3359+1)(99+1)}{84MHz} = 4ms$$

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled

Codice

```
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "string.h"
#include "stdio.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
typedef enum { KEY_IDLE, KEY_DEBOUNCE, KEY_SENT } KeyState;
// Per ogni tasto (4*4 = 16) ci possiamo torvare in 3 stati:
//     KEY_IDLE = il tasto non è premuto
//     KEY_DEBOUNCE = il tasto è premuto e contiamo per quanto tempo
//     KEY_SENT = il tasto è premuto per un tempo > DEBOUNCE, mandiamo via UART

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
#define N_ROWS 4
#define N_COLS 4
#define DEBOUNCE_MS 30

#define COL_PORT GPIOC
#define ROW_PORT GPIOC
```

```c
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim2;

UART_HandleTypeDef huart2;
DMA_HandleTypeDef hdma_usart2_tx;

/* USER CODE BEGIN PV */
volatile KeyState key_state[N_ROWS][N_COLS] = {{KEY_IDLE}}; // All'inizio assumiamo che nessun tasto sia premuto
volatile uint32_t press_start[N_ROWS][N_COLS] = {{0}};
volatile uint8_t  current_col = 0; // Tenere traccia di dove siamo arrivati a leggere

// Così possiamo iterarci sopra
const uint16_t COL_PIN[N_COLS] = {GPIO_PIN_8, GPIO_PIN_9, GPIO_PIN_10, GPIO_PIN_11};
const uint16_t ROW_PIN[N_ROWS] = {GPIO_PIN_12, GPIO_PIN_13, GPIO_PIN_2, GPIO_PIN_3};


const char keymap[N_ROWS][N_COLS] = {
    {'0','1','2','3'},
    {'4','5','6','7'},
    {'8','9','A','B'},
    {'C','D','E','F'}
};
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_DMA_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        // 1. leggo le righe con la colonna corrente
        for (int row = 0; row < N_ROWS; row++)
        {
            GPIO_PinState r = HAL_GPIO_ReadPin(ROW_PORT, ROW_PIN[row]);

            if (r == GPIO_PIN_RESET) // LOW -> tasto premuto
            {
                if (key_state[row][current_col] == KEY_IDLE)
                {
                    // Se ero in IDLE entro in DEBOUNCE
                   // Se non ero in IDLE non faccio nulla
                   key_state[row][current_col] = KEY_DEBOUNCE;
```

```
                    press_start[row][current_col] = HAL_GetTick(); // salvo il momento
in cui è stato premuto
                }

            }
            else // HIGH -> tasto non premuto
            {
                // Anche se ero in KEY_SENT, se non premo il tasto torno in IDLE
             key_state[row][current_col] = KEY_IDLE;
            }
        }

        // 2. Aggiorniamo la colonna
        HAL_GPIO_WritePin(COL_PORT, COL_PIN[current_col], GPIO_PIN_RESET); // Colonna
corrente ad alta impedenza (OFF)
        current_col = (current_col + 1) % N_COLS; // Prossima colonna
        HAL_GPIO_WritePin(COL_PORT, COL_PIN[current_col], GPIO_PIN_SET); // La porto a
gnd (ON)
    }
}
/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_DMA_Init();
  MX_USART2_UART_Init();
  MX_TIM2_Init();
  /* USER CODE BEGIN 2 */

  // Selezioniamo la prima colonna
  HAL_GPIO_WritePin(COL_PORT, COL_PIN[0], GPIO_PIN_SET);
  // Deselezioniamo le altre
  for (int i=1; i<N_COLS; i++){
        HAL_GPIO_WritePin(COL_PORT, COL_PIN[i], GPIO_PIN_RESET);
```

```
  }

  // Ora siamo pronti a far partire il Timer
  HAL_TIM_Base_Start_IT(&htim2);

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
        for (int row = 0; row < N_ROWS; row++)
            {
                for (int col = 0; col < N_COLS; col++)
                {
                    if (key_state[row][col] == KEY_DEBOUNCE &&
                        (HAL_GetTick() - press_start[row][col]) >= DEBOUNCE_MS)
                    {
                        // Il tasto è stato premuto oltre il DEBOUNCE
                     // Ci prepariamo all'invio
                     char buf[8];
                        int len = sprintf(buf, "%c\r\n", keymap[row][col]);
                        HAL_UART_Transmit(&huart2, (uint8_t*)buf, len, HAL_MAX_DELAY);
                        key_state[row][col] = KEY_SENT; // non reinviare finché non
torna KEY_IDLE
                    }
                }
            }
  }
  /* USER CODE END 3 */
}
```